CS1200 / CSCI E-120

Sorting algorithms and computational problems

Hesterberg–Vadhan textbook: Sections 2.5, 3.6 , 3.1 , 3.2 , Appendix A.3

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1:

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2:

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2: $((1, a), (2, b), (1, c), (2, d), (1, e), (2, f))$

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2: $((1, a), (2, b), (1, c), (2, d), (1, e), (2, f))$
- Group 3:

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2: $((1, a), (2, b), (1, c), (2, d), (1, e), (2, f))$
- Group 3: $((2^{\frac{1}{30}}, a), (1.001145, b), (10^{\frac{1}{100}}, c), (3^{\frac{1}{\pi^5}}, d), (4^{\frac{1}{70}}, e), (1.1001, f))$

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2: $((1, a), (2, b), (1, c), (2, d), (1, e), (2, f))$
- Group 3: $((2^{\frac{1}{30}}, a), (1.001145, b), (10^{\frac{1}{100}}, c), (3^{\frac{1}{\pi^5}}, d), (4^{\frac{1}{70}}, e), (1.1001, f))$
- Group 4:

A sorting exercise

Input: An array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$.[a]

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a *valid sorting* of A. That is, A' should be:

1. sorted by key values, i.e. $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$. and
2. a reordering of A, i.e. $\exists$ a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ for $i = 0, \ldots, n-1$.

- Group 1: $((3, a), (10^5, b), (50, c), (10^{20}, d), (17, e), (8, f))$
- Group 2: $((1, a), (2, b), (1, c), (2, d), (1, e), (2, f))$
- Group 3: $((2^{\frac{1}{30}}, a), (1.001145, b), (10^{\frac{1}{100}}, c), (3^{\frac{1}{\pi^5}}, d), (4^{\frac{1}{70}}, e), (1.1001, f))$
- Group 4: $((i, a), (2 + \frac{i}{2}, b), (-i, c), (1 + 3i, d), (1 - 3i, e), (2 + i, f))$

Sorting exercise: takeaways

- Definition matters: Group 4's input did not fall into our definition of 'Sorting Problem'.

Sorting exercise: takeaways

- Definition matters: Group 4's input did not fall into our definition of 'Sorting Problem'.
- Time to solve a problem depends on our definition of 'elementary operations' (Group 3's experience).

Sorting exercise: takeaways

- Definition matters: Group 4's input did not fall into our definition of 'Sorting Problem'.
- Time to solve a problem depends on our definition of 'elementary operations' (Group 3's experience).
- Some inputs can be sorted very easily (Group 2's input).

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example:

- $\mathrm{SORTING}$ problem:
  - $\mathcal{I} = \mathcal{O} =$

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example:

- $\mathrm{SORTING}$ problem:
  - $\mathcal{I} = \mathcal{O} =$ {All arrays of key-value pairs with keys in $\mathbb{R}$}
  - $f(x) =$

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example:

- $\mathrm{SORTING}$ problem:
  - $\mathcal{I} = \mathcal{O} =$ {All arrays of key-value pairs with keys in $\mathbb{R}$}
  - $f(x) =$ {All valid sortings of x}

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example:

- $\mathrm{SORTING}$ problem:
  - $\mathcal{I} = \mathcal{O} =$ {All arrays of key-value pairs with keys in $\mathbb{R}$}
  - $f(x) =$ {All valid sortings of x}
- Q: In what way would this definition change for the $\mathrm{SORTING-ON-COMPLEX-NUMBERS}$ problem?

Computational problem

A *computational problem* Π is a triple $(\mathcal{I}, \mathcal{O}, f)$ where:

- $\mathcal{I}$ is a (typically infinite) set of possible inputs (a.k.a. *instances*) x, and $\mathcal{O}$ is a (sometimes infinite) set of possible outputs y.
- For every input $x \in \mathcal{I}$, a *set* $f(x) \subseteq \mathcal{O}$ of *valid outputs* (a.k.a. *valid answers*).

Example:

- $\mathrm{SORTING}$ problem:
  - $\mathcal{I} = \mathcal{O} =$ {All arrays of key-value pairs with keys in $\mathbb{R}$}
  - $f(x) =$ {All valid sortings of x}
- Q: In what way would this definition change for the $\mathrm{SORTING-ON-COMPLEX-NUMBERS}$ problem?
  - $\mathbb{R} \to \mathbb{C}$
  - $f(x) =$ {All valid sortings of x} (if all keys are real), the empty set ϕ (otherwise)

Algorithm

- (Informal definition): An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.

Algorithm

- (Informal definition): An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.
- Algorithm A *solves* computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if the following holds:
  1. For every input $x \in \mathcal{I}$ with $f(x) \neq \emptyset$, we have $A(x) \in f(x)$.

Algorithm

- (Informal definition): An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.
- Algorithm A *solves* computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if the following holds:
  1. For every input $x \in \mathcal{I}$ with $f(x) \neq \emptyset$, we have $A(x) \in f(x)$.
  2. There is a special output $\perp \notin \mathcal{O}$ such that for every input $x \in \mathcal{I}$ with $f(x) = \phi$, we have $A(x) = \perp$.

Algorithm

- (Informal definition): An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.
- Algorithm A *solves* computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if the following holds:
  1. For every input $x \in \mathcal{I}$ with $f(x) \neq \emptyset$, we have $A(x) \in f(x)$.
  2. There is a special output $\perp \notin \mathcal{O}$ such that for every input $x \in \mathcal{I}$ with $f(x) = \phi$, we have $A(x) = \perp$.

Q: What should Group 4 have responded?

Algorithm

- (Informal definition): An *algorithm* is a well-defined "procedure" A for "transforming" any input x into an output $A(x)$.
- Algorithm A *solves* computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if the following holds:
  1. For every input $x \in \mathcal{I}$ with $f(x) \neq \emptyset$, we have $A(x) \in f(x)$.
  2. There is a special output $\perp \notin \mathcal{O}$ such that for every input $x \in \mathcal{I}$ with $f(x) = \phi$, we have $A(x) = \perp$.

Q: What should Group 4 have responded?

Note: *A fundamental point in the theory of algorithms is that we distinguish between computational problems and algorithms that solve them*

Measuring efficiency

- We first define a size parameter, a function $\mathrm{size} : \mathcal{I} \to \mathrm{R}^{\geq 0}$.
  - Example: for $\mathrm{SORTING}$ problem, $\mathrm{size}(x)$ is number of key-value pairs.

Measuring efficiency

- We first define a size parameter, a function $\mathrm{size} : \mathcal{I} \to \mathrm{R}^{\geq 0}$.
  - Example: for $\mathrm{SORTING}$ problem, $\mathrm{size}(x)$ is number of key-value pairs. Is it a reasonable definition (think in terms of Group 3's experience)?

Measuring efficiency

- We first define a size parameter, a function $\mathrm{size} : \mathcal{I} \to \mathrm{R}^{\geq 0}$.
  - Example: for $\mathrm{SORTING}$ problem, $\mathrm{size}(x)$ is number of key-value pairs. Is it a reasonable definition (think in terms of Group 3's experience)?
- For an algorithm A, an input set $\mathcal{I}$, and input size function $\mathrm{size} : \mathcal{I} \to \mathbb{N}$, the *(worst-case) running time* of A is :

Measuring efficiency

- We first define a size parameter, a function $\mathrm{size} : \mathcal{I} \to \mathrm{R}^{\geq 0}$.
  - Example: for $\mathrm{SORTING}$ problem, $\mathrm{size}(x)$ is number of key-value pairs. Is it a reasonable definition (think in terms of Group 3's experience)?
- For an algorithm A, an input set $\mathcal{I}$, and input size function $\mathrm{size} : \mathcal{I} \to \mathbb{N}$, the *(worst-case) running time* of A is :

$$T(n) = \max_{x \in \mathcal{I} : \mathrm{size}(x) \leq n} (\# \text{ "basic operations" performed by } A \text{ on input } x) .$$

Measuring efficiency

- We first define a size parameter, a function $\mathrm{size} : \mathcal{I} \to \mathrm{R}^{\geq 0}$.
  - Example: for $\mathrm{SORTING}$ problem, $\mathrm{size}(x)$ is number of key-value pairs. Is it a reasonable definition (think in terms of Group 3's experience)?
- For an algorithm A, an input set $\mathcal{I}$, and input size function $\mathrm{size} : \mathcal{I} \to \mathbb{N}$, the *(worst-case) running time* of A is :

$$T(n) = \max_{x \in \mathcal{I} : \mathrm{size}(x) \leq n} (\# \text{ "basic operations" performed by } A \text{ on input } x) .$$

The *(worst-case) running time for fixed-size inputs* of A :

$$T^{=}(n) = \max_{x : \mathrm{size}(x) = n} (\# \text{ "basic operations" performed by } A \text{ on input } x) .$$

Measuring efficiency: Remarks

- Basic operations: arithmetic on individual numbers, manipulation of pointers, and writing/reading individual numbers to/from memory.

Measuring efficiency: Remarks

- Basic operations: arithmetic on individual numbers, manipulation of pointers, and writing/reading individual numbers to/from memory.
  Q: Should the Python function `A.sort()` count as a basic operation?

Measuring efficiency: Remarks

- Basic operations: arithmetic on individual numbers, manipulation of pointers, and writing/reading individual numbers to/from memory.
  Q: Should the Python function `A.sort()` count as a basic operation?
- Worst-case runtime: general-purpose and application-independent guarantees. If an algorithm has good performance on some inputs and not on others, we may need think hard about which kinds of inputs arise in our application before using it.

Measuring efficiency: Remarks

- Basic operations: arithmetic on individual numbers, manipulation of pointers, and writing/reading individual numbers to/from memory.
  Q: Should the Python function `A.sort()` count as a basic operation?
- Worst-case runtime: general-purpose and application-independent guarantees. If an algorithm has good performance on some inputs and not on others, we may need think hard about which kinds of inputs arise in our application before using it.
- Other notions of efficiency: how much space an algorithm needs; energy efficiency in the age of GenAI.

MergeSort

Solves the SORTING problem.

```
MergeSort(A):
Input  : An array A = ((K_0, V_0), ..., (K_{n-1}, V_{n-1})), where each K_i ∈ ℝ
Output : A valid sorting of A
0 if n ≤ 1 then return A;
1 else
2 |   i = ⌈n/2⌉
3 |   B = MergeSort(((K_0, V_0), ..., (K_{i-1}, V_{i-1})))
4 |   B' = MergeSort(((K_i, V_i), ..., (K_{n-1}, V_{n-1})))
5 |   return Merge (B,B')
```

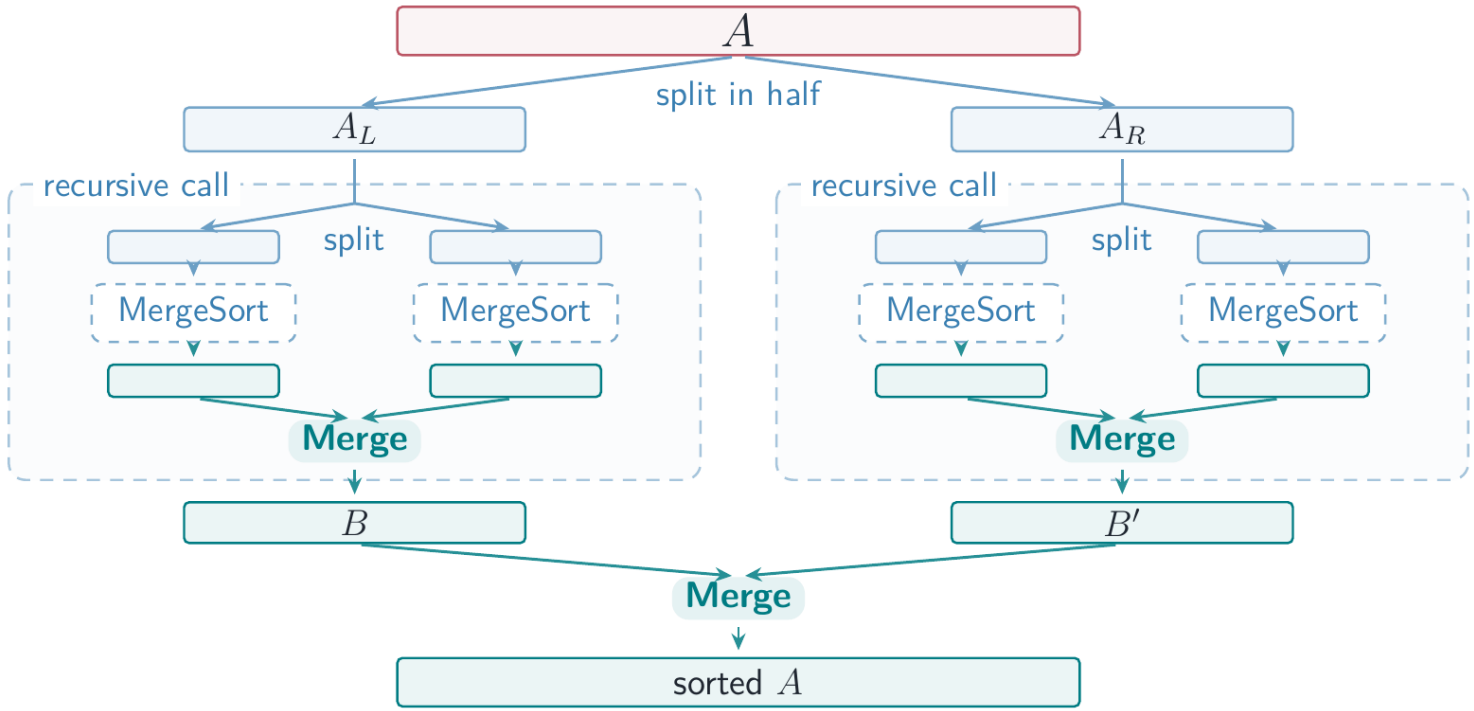
MergeSort

Solves the SORTING problem.

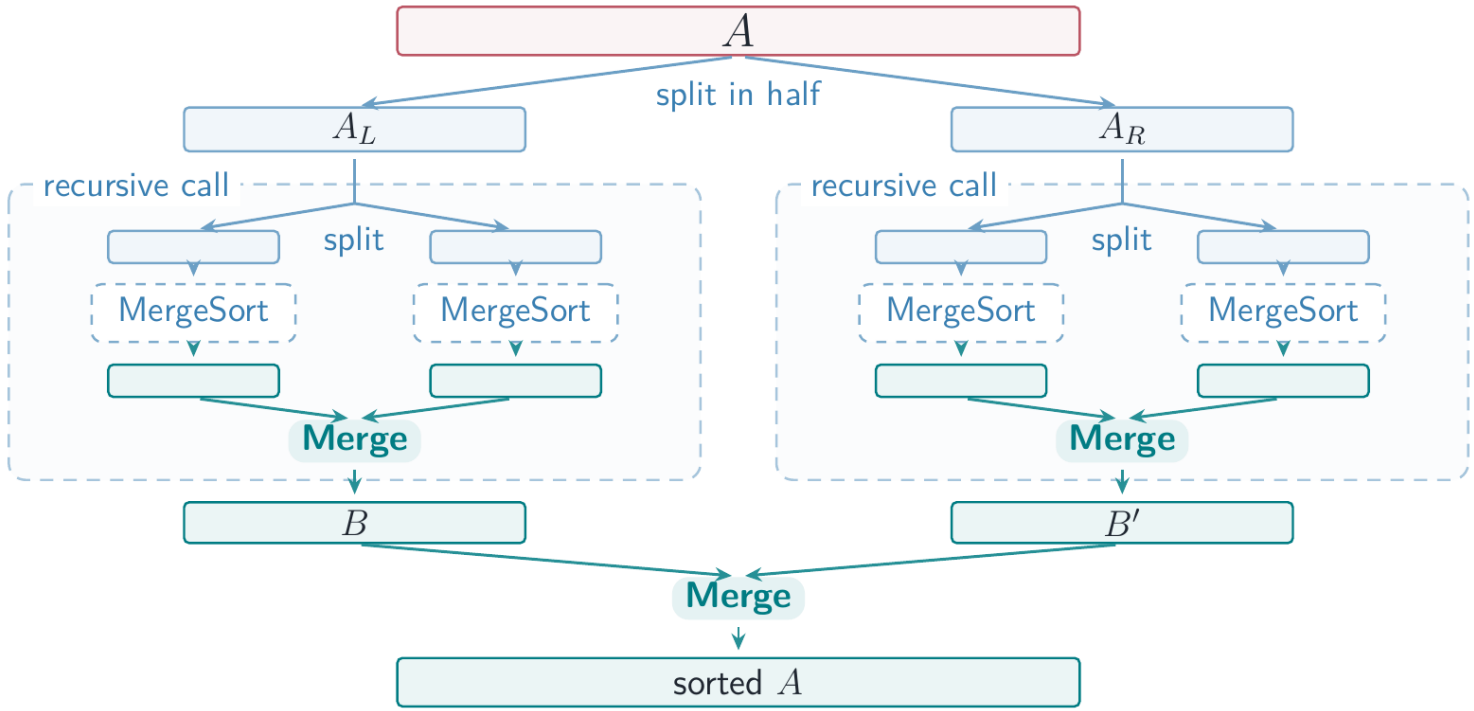
```
MergeSort(A):
Input  : An array A = ((K_0, V_0), ..., (K_{n-1}, V_{n-1})), where each K_i ∈ ℝ
Output : A valid sorting of A
0 if n ≤ 1 then return A;
1 else
2 |   i = ⌈n/2⌉
3 |   B = MergeSort(((K_0, V_0), ..., (K_{i-1}, V_{i-1})))
4 |   B' = MergeSort(((K_i, V_i), ..., (K_{n-1}, V_{n-1})))
5 |   return Merge (B,B')
```

MergeSort makes $O(n \log n)$ comparisons (next lecture).

MergeSort


A
split in half
A_L
A_R
recursive call
split
MergeSort
MergeSort
Merge
recursive call
split
MergeSort
MergeSort
Merge
B
B'
Merge
sorted A

MergeSort



Proof of correctness: using strong induction (see Theorem 2.7 in Hesterberg-Vadhan).

Sorting on Finite Universe

Input: A universe size $U \in \mathbb{N}$ and an array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in [U]$

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a valid sorting of A.

Computational Problem SORTING ON FINITE UNIVERSE()

Sorting on Finite Universe

Input: A universe size $U \in \mathbb{N}$ and an array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in [U]$

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a valid sorting of A.

Computational Problem SORTING ON FINITE UNIVERSE()

Q: Why is it different from the SORTING problem?

Sorting on Finite Universe

Input: A universe size $U \in \mathbb{N}$ and an array A of key-value pairs $((K_0, V_0), \ldots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in [U]$

Output: An array A' of key-value pairs $((K'_0, V'_0), \ldots, (K'_{n-1}, V'_{n-1}))$ that is a valid sorting of A.

Computational Problem SORTING ON FINITE UNIVERSE()

Q: Why is it different from the SORTING problem?

Singleton Bucket Sort (also called Pigeonhole sort): an algorithm to solve SORTINGONFINITEUNIVERSE in time $O(n + U)$.

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$
- A simple way to sort is to 'put' all the key-value pairs with key $= 1$ at one place, all the key-value pairs with key $= 2$ at another place, and then output them in the right order.

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$
- A simple way to sort is to 'put' all the key-value pairs with key $= 1$ at one place, all the key-value pairs with key $= 2$ at another place, and then output them in the right order.

Algorithm (input an array of key-value pairs with real keys):

- Initialize an array C of length U to have zeroes in every entry.

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$
- A simple way to sort is to 'put' all the key-value pairs with key $= 1$ at one place, all the key-value pairs with key $= 2$ at another place, and then output them in the right order.

Algorithm (input an array of key-value pairs with real keys):

- Initialize an array C of length U to have zeroes in every entry.
- Make a pass over the array A of key-value pairs.
- Add $A[i]$ to the head of the *linked list* at $C(A[i][0])$.

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$
- A simple way to sort is to 'put' all the key-value pairs with key $= 1$ at one place, all the key-value pairs with key $= 2$ at another place, and then output them in the right order.

Algorithm (input an array of key-value pairs with real keys):

- Initialize an array C of length U to have zeroes in every entry.
- Make a pass over the array A of key-value pairs.
- Add $A[i]$ to the head of the *linked list* at $C(A[i][0])$.
- Construct the output by traversing the linked lists $C[0], C[1], \ldots$. Traverse each linked list from tail to head.

Singleton Bucket Sort

- Recall group 2's input: $A = ((2, a), (1, b), (2, c), (1, d), (2, e), (1, f))$
- A simple way to sort is to 'put' all the key-value pairs with key $= 1$ at one place, all the key-value pairs with key $= 2$ at another place, and then output them in the right order.

Algorithm (input an array of key-value pairs with real keys):

- Initialize an array C of length U to have zeroes in every entry.
- Make a pass over the array A of key-value pairs.
- Add $A[i]$ to the head of the *linked list* at $C(A[i][0])$.
- Construct the output by traversing the linked lists $C[0], C[1], \ldots$. Traverse each linked list from tail to head.

Stability: sorted output maintains the ordering of Values (if there is an ordering).

Singleton Bucket Sort: $U = 5$

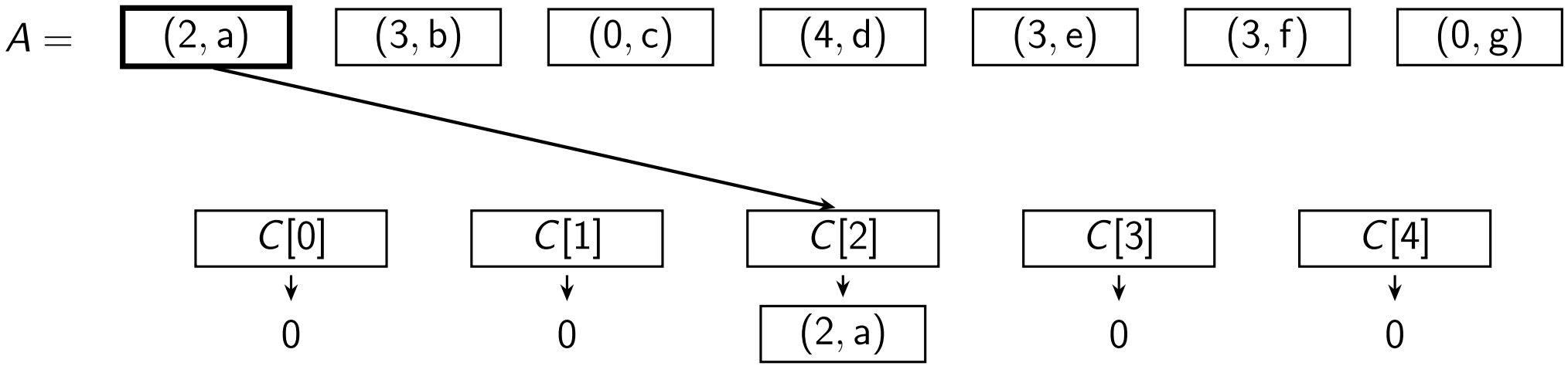
Initialize all five buckets to 0 (null).

$A =$ | $(2, \text{a})$ | $(3, \text{b})$ | $(0, \text{c})$ | $(4, \text{d})$ | $(3, \text{e})$ | $(3, \text{f})$ | $(0, \text{g})$ |

$C[0]$	$C[1]$	$C[2]$	$C[3]$	$C[4]$
↓	↓	↓	↓	↓
0	0	0	0	0

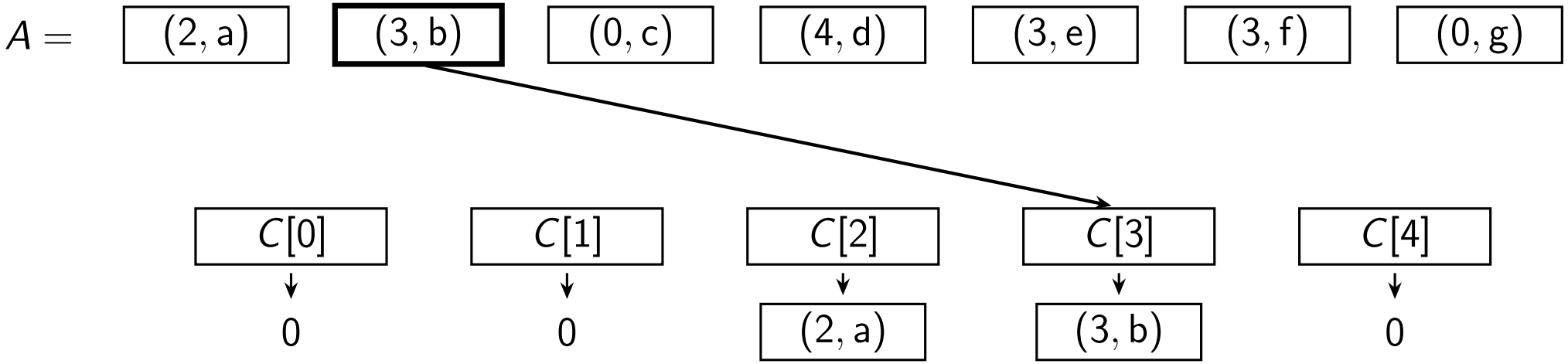
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



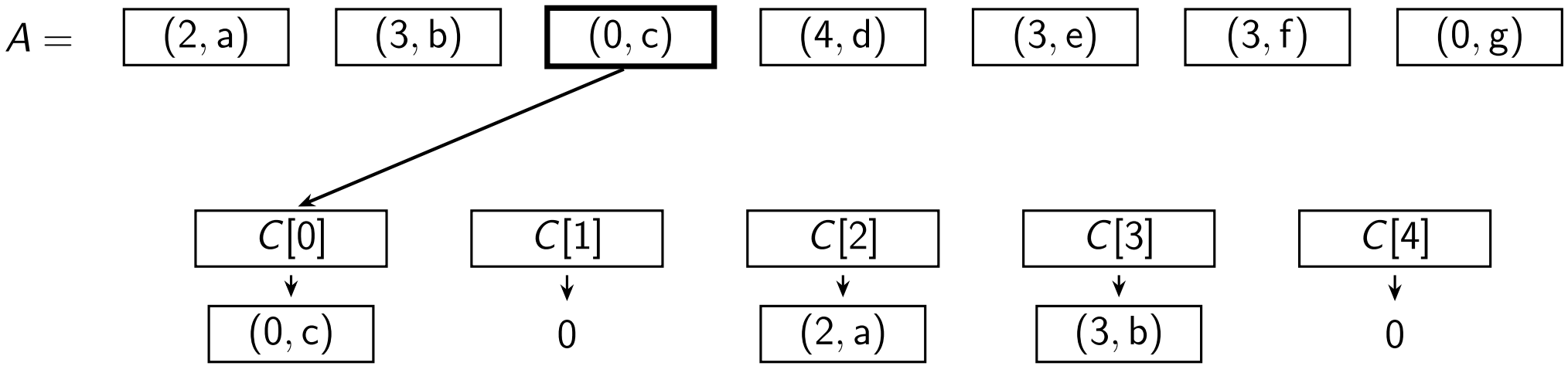
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



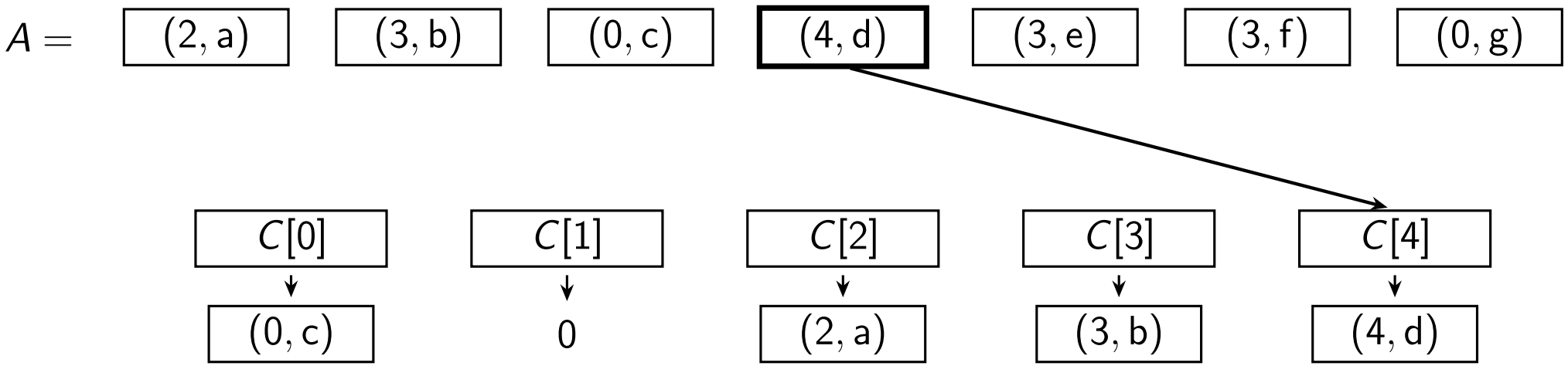
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



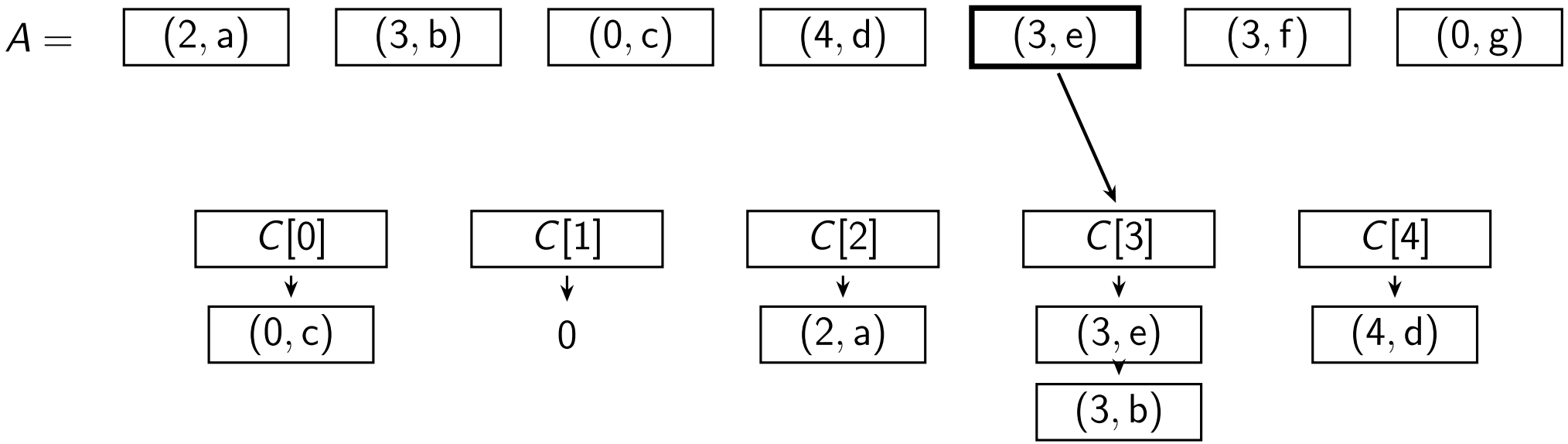
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



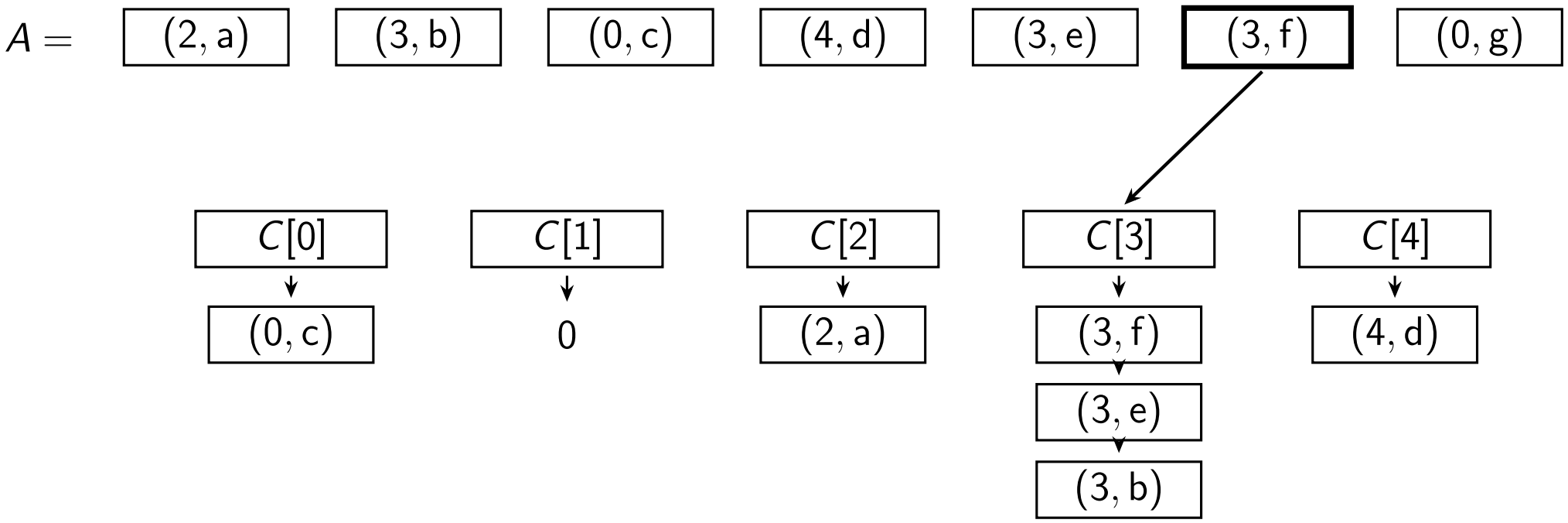
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



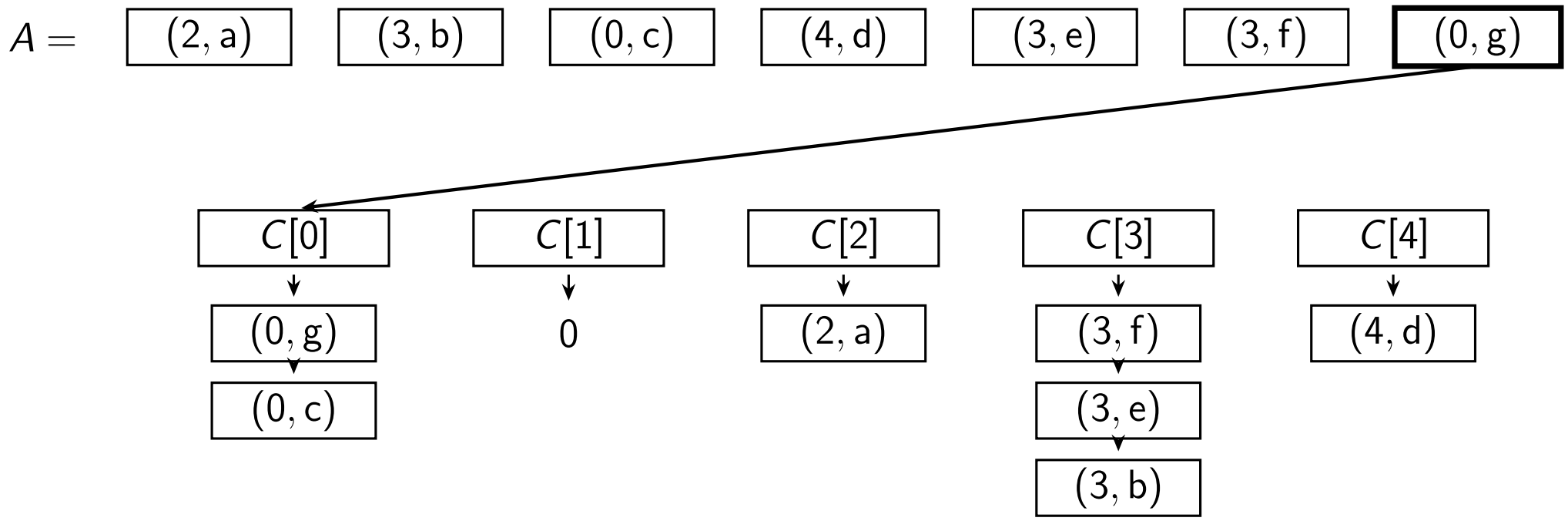
Singleton Bucket Sort: $U = 5$

Read A from left to right; insert at the head (top) of the list.



Singleton Bucket Sort: $U = 5$

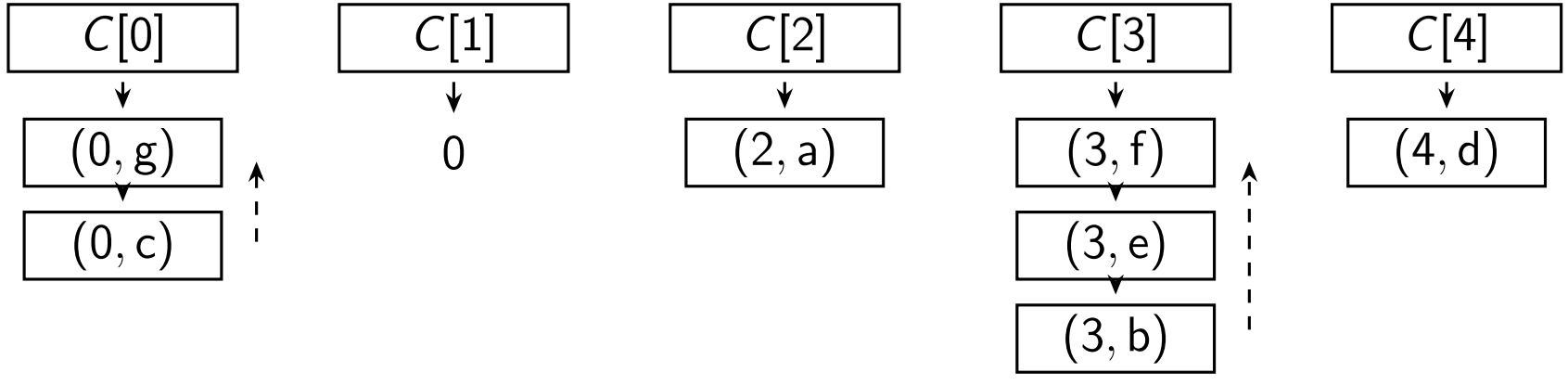
Read A from left to right; insert at the head (top) of the list.



Singleton Bucket Sort: $U = 5$

All records inserted. Lists are shown from head to tail, top to bottom.

$A =$ $(2, \mathrm{a})$ $(3, \mathrm{b})$ $(0, \mathrm{c})$ $(4, \mathrm{d})$ $(3, \mathrm{e})$ $(3, \mathrm{f})$ $(0, \mathrm{g})$



Visit $C[0], \ldots, C[4]$ in order; traverse each list from tail to head.

output: $(0, \mathrm{c})$ $(0, \mathrm{g})$ $(2, \mathrm{a})$ $(3, \mathrm{b})$ $(3, \mathrm{e})$ $(3, \mathrm{f})$ $(4, \mathrm{d})$